

Basic (plug search in as a plain tool first) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# search_tool_integration_practice.py – Basic section
from langchain_core.tools import tool
from retriever import retrieve

@tool
def search_docs(query):
    """Search the document set for relevant text."""
    results = retrieve(query)
    text = ""
    for chunk in results:
        text = text + chunk.page_content + "\n\n"
    return text

# wire it into the graph's model node
model_with_tools = model.bind_tools([search_docs])

# test it
response = graph.invoke({"messages": [("user", "What does the document say about
print(response["messages"][-1])
```

This works. It's missing type hints, and it builds the joined text with a loop instead of a cleaner one-liner — both fine for a first version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints, a real docstring, and an explicit "no results" message

```
# search_tool_integration_practice.py – Basic section
from langchain_core.tools import tool
from langgraph.graph import StateGraph
from retriever import retrieve

@tool
def search_docs(query: str) -> str:
    """Search the document set for text relevant to the query.

    Use this whenever the user's question could be answered by
    looking something up in the project's own documents.
    """
    results = retrieve(query)
    if not results:
        return "No relevant documents found."
    return "\n\n".join(chunk.page_content for chunk in results)

# wire it into the graph's model node
model_with_tools = model.bind_tools([search_docs])

def call_model(state: dict) -> dict:
    response = model_with_tools.invoke(state["messages"])
    return {"messages": [response]}

# test it
result = graph.invoke({"messages": [("user", "What does the document say about va
print(result["messages"][-1])
```

Difference from Basic: full type hints, and a more descriptive docstring that explicitly tells the model *when* to use this tool — the model relies entirely on this text to decide, so being specific here directly improves how reliably it gets called at the right time. An explicit "no results" message instead of an empty string, so the model gets a clear signal instead of silence it might misread as an error. A

generator expression (`"\n\n".join(...)`) instead of a manual loop — same result, no intermediate variable to manage. This version still trusts `retrieve()` never to raise, and never to return more chunks than the model should have to read in one turn — that's what Advanced adds.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — caught failures and a capped result count

```
# search_tool_integration_practice.py – Basic section
from langchain_core.tools import tool
from retriever import retrieve

MAX_RESULTS = 5

@tool
def search_docs(query: str) -> str:
    """Search the document set for text relevant to the query.

    Use this whenever the user's question could be answered by
    looking something up in the project's own documents.
    """
    try:
        results = retrieve(query)
    except Exception as exc:
        return f"Search is temporarily unavailable: {exc}"

    if not results:
        return "No relevant documents found."

    capped_results = results[:MAX_RESULTS]
    return "\n\n".join(chunk.page_content for chunk in capped_results)

model_with_tools = model.bind_tools([search_docs])
result = graph.invoke({"messages": [("user", "What does the document say about va
print(result["messages"][-1])
```

A failed `retrieve()` call now hands the model a short, honest sentence instead of crashing the whole turn with a traceback — the model can decide to tell the user search is down, retry with a rephrased query, or answer from what it already knows, instead of the graph run failing outright. Capping to `MAX_RESULTS` chunks keeps one tool call from consuming an outsized share of the model's context window on a larger document set. Like `MAX_ATTEMPTS` in `search_failure`, `MAX_RESULTS` is written as a plain constant here for readability — in a real deployment it's a

`config.py` setting, not a hardcoded literal, since how many chunks is "too many" depends on the model's context window and changes as you swap models.

Approach 2 — returning sources separately, for a later approval step

The Intermediate version's return value is a single joined string — good enough for the model to read, but it throws away exactly which chunks were used. `approval_pause` needs a human reviewer to see the actual sources, not a paraphrase, so this approach keeps them alongside the text instead of discarding them.

```
# search_tool_integration_practice.py – Basic section
from dataclasses import dataclass
from langchain_core.tools import tool
from retriever import retrieve

MAX_RESULTS = 5

@dataclass
class SearchResult:
    text: str
    sources: list[str]

def search_docs_with_sources(query: str) -> SearchResult:
    """Search the document set and keep track of which chunks were used."""
    try:
        results = retrieve(query)
    except Exception as exc:
        return SearchResult(text=f"Search is temporarily unavailable: {exc}", sources=[])

    if not results:
        return SearchResult(text="No relevant documents found.", sources=[])

    capped_results = results[:MAX_RESULTS]
    text = "\n\n".join(chunk.page_content for chunk in capped_results)
    sources = [chunk.metadata.get("source", "unknown") for chunk in capped_results]
    return SearchResult(text=text, sources=sources)

@tool
def search_docs(query: str) -> str:
    """Search the document set for text relevant to the query."""
    return search_docs_with_sources(query).text
```

`search_docs` (the `@tool`) is still what the model calls and reads — that part of the contract doesn't change. `search_docs_with_sources` is the same logic, used directly by a graph node (not through the model) when the node itself, not the model, needs to know exactly which sources were used — for example a `search_node` in the full Build Task graph, which saves `sources` into state for `approval_node` to show a human later.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's tool does real, correct work on the happy path, but an unhandled `retrieve()` failure crashes the turn, and there's no limit on how much text one call can return. Approach 1 fixes both of those directly inside the `@tool` function, with no new dependencies or structure — the smallest useful upgrade. Approach 2 solves a different problem: it keeps the *sources* (not just the joined text) available to the rest of the graph, which only matters once something downstream — like a human approval step — needs to show exactly what was searched, not just what the model was told.

Which one should you actually write? Approach 1's caught-failure-plus-cap is worth adding to every tool that wraps an external lookup, in any project — it costs a few lines and prevents a whole class of confusing crashes. Approach 2's separate `sources` tracking is only worth the extra structure once something else in your graph — an approval step, a citation feature, a "show your work" UI — actually needs the sources, not just the model. For this exercise alone, Approach 1 is enough; Approach 2 is what you'll reach for once you build the full Build Task graph.